

# How to Actually Run AI Evals: Rubrics, Golden Datasets & LLM-as-a-Judge

Walk the full evaluation loop end to end — write the rubric, hand-label a golden dataset, build an LLM judge, and prove the judge agrees with you.

For product managers and engineers shipping an LLM feature who need to know whether it is actually good enough to deploy · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

## What matters

- ✓ There are four kinds of eval: code-based assertions, human labels, LLM-as-a-judge, and user/business signals. They form a ladder of scale, and each one is validated by the one before it.
- ✓ The golden dataset is the foundation. If your hand-labelled examples are bad, every automated eval built on them is bad, no matter how sophisticated the tooling.
- ✓ A rubric turns 'is this good?' into named dimensions with written definitions — for a support agent: product knowledge, policy compliance, and tone, each graded good/average/bad.
- ✓ Start with about 10 labelled rows for internal iteration; get to 100 or more before you trust a number for production.
- ✓ More data means more confidence but slower iteration. These are orthogonal dimensions and you must consciously pick a point between them.
- ✓ An LLM judge must output an explanation, not just a score, so you can audit its reasoning rather than trusting a bare label.
- ✓ Match rate — how often the judge's label equals the human's label — is what tells you whether the judge can be trusted. A judge that scores everything 'good' has a great average and zero value.
- ✓ Disagreement between teammates during labelling is a feature: it surfaces genuine product decisions the specification never settled.

## What it covers

### 01 Why evaluation became the defining product skill 1:22

LLMs hallucinate, so the only way to know your product works is to measure it — which is why the people selling you the models tell you to check the models' answers.

### 02 The four types of eval 2:44

Code-based, human, LLM-as-a-judge, and user evals form a ladder from cheap and narrow to expensive and real.

### 03 Step one: the prompt and the context it needs 7:30

Before you can evaluate anything you need a system to evaluate — and the prompt's quality is mostly determined by the context you feed it.

### 04 Step two: write the rubric before you grade anything 15:08

Turn 'is this good?' into named dimensions with written definitions, so that two people grading the same output reach the same label for the same reason.

- 05 **Step three: build the golden dataset by hand** 16:30  
Run real questions through the system, put inputs and outputs in a spreadsheet, and grade each row against the rubric with your team.
- 06 **What you find when you actually look at the data** 22:44  
Real failures are specific and often boring — like a model asserting that 45 minutes is past a 60-minute window.
- 07 **How many examples do you actually need?** 24:49  
Ten rows to start iterating internally, a hundred or more before you trust the number for production — and the answer is fundamentally a statistics question.
- 08 **Step four: encode the rubric as an LLM judge** 32:21  
The judge prompt is your rubric, rewritten as instructions to a model, returning a label and — critically — an explanation.
- 09 **Step five: judge the judge with match rate** 36:29  
Compare the judge's labels against your human labels and measure how often they agree — because a judge that scores everything 'good' looks great and tells you nothing.
- 10 **Shipping: internal, then a small slice, then real users** 47:34  
Offline evals get you to a candidate; dogfooding and a small A/B slice tell you whether it works in the world — and user signals are noisier than they look.

## Glossary

**Eval** — A systematic measurement of one specific aspect of an AI system's quality, such as whether a reply followed policy.

**Rubric** — The named dimensions you grade on, each with written definitions of what good, average, and bad mean, so different people label consistently.

**Match rate** — The fraction of examples where the LLM judge's label equals the human's label. It measures whether the judge can be trusted.

**Code-based eval** — A deterministic pass/fail check written in ordinary code, such as verifying a required string is present or a field holds an allowed value.

**Dogfooding** — Using your own product internally before exposing it to customers, so the people who know what good looks like find the problems first.

**Golden dataset** — A curated set of example inputs with human-assigned quality labels, used as the reference standard for measuring the system and validating automated judges.

**LLM-as-a-judge** — Using a language model to apply your rubric to your system's outputs at scale, producing a label and an explanation the way a human grader would.

**Alignment (of a judge)** — The state of an LLM judge agreeing with human labels often enough that its scores can be believed without re-checking each one.

**User eval** — Quality signal gathered from real users in production — explicit thumbs up/down, complaints, or downstream business metrics.

Explanations are AI-generated. Check anything you intend to rely on.